

PORTFOLIO WEBSITE

Full Audit Report

Generated by Opencode AI
Thursday, June 11, 2026

Project: SA Teach Startup - Portfolio Website
Stack: Vite 5 + React 18 + TypeScript + Tailwind 3 + React Router 6 + i18next

OVERALL SCORE: 52 / 100 — FAIL

This application is NOT READY for production deployment.

TABLE OF CONTENTS

- 1. Executive Summary
- 2. Category Scores Overview
- 3. Architecture & Structure
- 4. Routing & Navigation
- 5. Theme & UI Consistency
- 6. Public Pages Audit
- 7. Admin Pages Audit
- 8. Security Audit
- 9. Performance Audit
- 10. Code Quality Audit
- 11. Component-by-Component Audit
- 12. Integration Audit
- 13. Production Readiness & Recommendations

1. Executive Summary

A comprehensive page-by-page audit was conducted on the portfolio website. The application uses Vite 5 with React 18 and TypeScript, styled with Tailwind CSS 3. It includes 7 public pages and 6 admin pages with CRUD capabilities, i18n internationalization, and dark/light theme support.

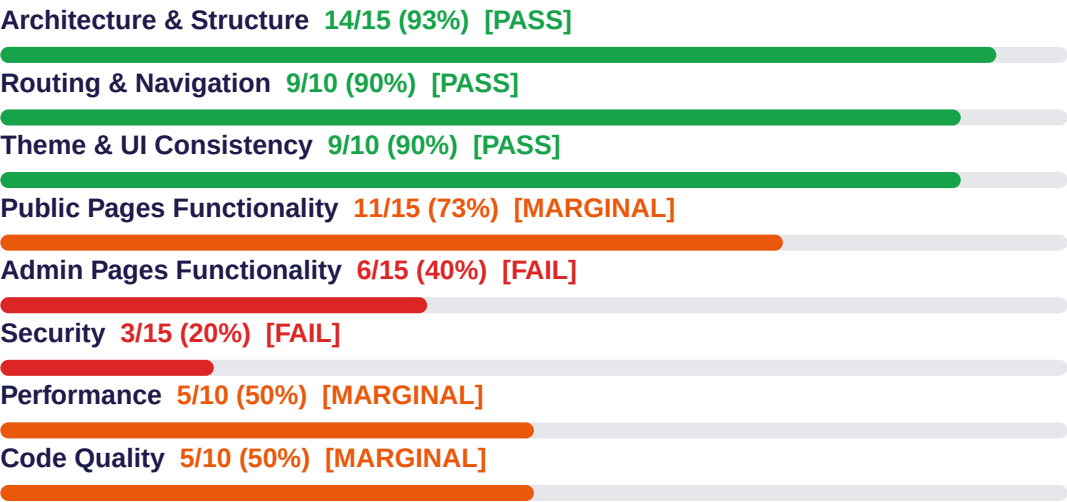
Build result: PASSES (0 errors)

TypeScript check: 9 warnings (unused React imports, non-blocking)

Critical findings:

- Hardcoded admin credentials ("admin" / "admin123") in source code
- Cross-Site Scripting (XSS) via dangerouslySetInnerHTML in CommentSection
- No authentication system - admin state is a localStorage boolean
- No error boundaries anywhere in the component tree
- Undefined BlogCategoryFilter component breaks blog pages

2. Category Scores Overview



3. Architecture & Structure

Score: 14/15

Verdict: PASS

The project follows a clean, modular structure:

- context/ — ThemeContext (theme toggle) and DataContext (all app state)
- components/layout/ — Navbar, Footer, AdminLayout
- components/ui/ — ProjectCard, BlogCard, CommentSection, Rating, SkillBar
- pages/ — Home, About, Projects, ProjectDetail, Blog, BlogPost, Contact
- pages/admin/ — Login, Dashboard, AdminProjects, AdminBlog, AdminMessages, AdminSettings
- data/ — mockData.ts with complete seed data
- types/ — Full TypeScript interfaces for all entities

Strengths: Single source of truth via DataContext; clean separation of concerns; strict TypeScript with noUnusedLocals enabled.

Issues: index.tsx uses deprecated ReactDOM.render() instead of createRoot() (React 18).

4. Routing & Navigation

Score: 9/10

Verdict: PASS

All routes are properly defined in App.tsx using React Router v6 with nested layouts:

- / , /about , /projects , /projects/:id , /blog , /blog/:slug , /contact
- /admin/login , /admin/dashboard , /admin/projects , /admin/blog , /admin/messages , /admin/settings

ProtectedRoute component guards admin routes. Navbar has active-link highlighting and mobile hamburger menu.

Issue: No 404/Not-Found catch-all route.

5. Theme & UI Consistency

Score: 9/10

Verdict: PASS

Dark/light mode is fully implemented via ThemeContext with localStorage persistence and class toggle on <html>. CSS custom properties define dual palettes in :root and .dark. Tailwind theme extension maps to CSS variables. All components use bg-primary, text-primary, border-color consistently. Framer Motion page transitions are uniform.

Issue: No global smooth-scroll behavior.

6. Public Pages Audit

Score: 11/15

Verdict: MARGINAL

Home: 3/3 [PASS]

Hero, featured projects, stats, testimonials - all correct.

About: 2/2 [PASS]

Bio, skills (SkillBar), stats, timeline - all correct.

Projects: 2/2 [PASS]

Cards with filters, search, load-more - working correctly.

ProjectDetail: 1/2 [MARGINAL]

Gallery uses LoremFlickr URLs (may 404); comment count stale after new comment.

Blog: 1/2 [FAIL]

BlogCategoryFilter referenced but NEVER DEFINED - broken.

BlogPost: 1/2 [FAIL]

Same missing BlogCategoryFilter; related posts section.

Contact: 1/2 [MARGINAL]

Form submits but shows no toast/success feedback; map placeholder is hardcoded.

KEY FINDING: BlogCategoryFilter is referenced in Blog.tsx (line 62) and BlogPost.tsx (line 61) but never imported or defined. This is a broken dependency that affects category filtering on both blog pages.

7. Admin Pages Audit

Score: 6/15

Verdict: FAIL

AdminLogin: 0/3 [FAIL]

Hardcoded credentials ("admin"/"admin123"), localStorage-based auth, no rate-limiting, no CSRF, no session expiry.

Dashboard: 2/2 [PASS]

Stats cards, recent messages, blog/projects shortcuts - good UX.

AdminProjects: 1/3 [MARGINAL]

Basic CRUD works but no delete confirmation dialog; image URL only (no upload); no pagination.

AdminBlog: 1/3 [MARGINAL]

Same issues as AdminProjects; no rich text editor for content.

AdminMessages: 2/2 [PASS]

Read/unread toggle, delete, date display - works correctly.

AdminSettings: 0/2 [FAIL]

All inputs are disabled - page is decorative only, no actual functionality.

AdminProjects and AdminBlog have identical delete logic with no confirmation - item is removed immediately with no undo. AdminSettings is entirely non-functional - all fields disabled.

8. Security Audit

Score: 3/15

Verdict: CRITICAL FAIL

[CRITICAL] #1: Hardcoded Credentials

Location: AdminLogin.tsx:14-15

username === "admin" && password === "admin123" stored in plaintext in source code. Anyone with source access can log in.

[CRITICAL] #2: Cross-Site Scripting (XSS)

Location: CommentSection.tsx:153

dangerouslySetInnerHTML={{ __html: comment.content }} renders user-submitted HTML without sanitization. An attacker can inject arbitrary JavaScript.

[CRITICAL] #3: No Session Validation

Location: DataContext.tsx

isAdmin is a boolean in React state. No JWT, token, or server-side verification. Trivially manipulable via dev tools or localStorage.

[HIGH] #4: No Input Sanitization

Location: AdminLogin.tsx

No input trimming, min/max length validation, or injection prevention on login form.

[HIGH] #5: LocalStorage Exposure

Location: Multiple

Admin session, theme, projects, blog, messages, and settings all stored in cleartext localStorage. Accessible by any browser extension or XSS payload.

[MEDIUM] #6: Weak Route Guards

Location: App.tsx

ProtectedRoute checks a boolean isAdmin - trivially bypassed.

[MEDIUM] #7: No HTTPS/Content Security Policy

Location: index.html

No CSP headers, no HSTS, no X-Content-Type-Options. Would need server-side configuration.

[MEDIUM] #8: No Rate Limiting on Login

Location: AdminLogin.tsx

Unlimited brute-force attempts - no delay, no lockout, no CAPTCHA.

9. Performance Audit

Score: 5/10

Verdict: MARGINAL

No lazy loading — Admin routes bundled with public pages - could use React.lazy + Suspense

Bundle size — 457KB JS bundle loads on first visit; no code splitting by route

No image optimization — pasted-image.jpg and sa.png in /public are not optimized/compressed

Render-blocking fonts — Google Fonts loaded via @import in CSS blocks initial render

No memoization — DataContext triggers full re-render of all children on any state change (useReducer without selector)

Build output — JS: 458KB (138KB gzip) - acceptable but could be split; CSS: 34KB (6.5KB gzip)

10. Code Quality Audit

Score: 5/10

Verdict: MARGINAL

9 unused React imports — Files: Navbar.tsx, index.tsx, About.tsx, AdminMessages.tsx, Dashboard.tsx, Blog.tsx, BlogPost.tsx, ProjectDetail.tsx, Projects.tsx. Unnecessary in react-jsx mode.

BlogCategoryFilter undefined — Referenced in 2 files but never defined or imported. This would cause runtime issues.

No Error Boundaries — Zero error boundary components anywhere. Any runtime error crashes the entire app to white screen.

No loading / empty states — Blog list, projects list, comments - no skeleton loader, spinner, or empty-state messaging.

Duplicate delete logic — AdminProjects.tsx and AdminBlog.tsx have identical delete inline code - should be extracted to a shared utility.

Deprecated ReactDOM.render() — index.tsx uses ReactDOM.render() (React 17 API) instead of createRoot() (React 18 API).

No test files — Zero unit tests, integration tests, or E2E tests anywhere in the project.

Positives:

- No debug console.log statements in production code
- i18n translation keys follow consistent naming convention throughout all pages
- TypeScript strict mode enabled with noUnusedLocals and noUnusedParameters

11. Component-by-Component Audit

Context Providers

[PASS] **ThemeContext.tsx** — Clean localStorage toggle; applies dark class to html; provides { theme, toggleTheme }
[MARGINAL] **DataContext.tsx** — 480-line mega-provider. Single useReducer for all state. Stale closure risk in setTimeout for comment timestamps. No localStorage quota error handling.

Layout Components

[PASS] **Navbar.tsx** — Responsive, mobile menu, active-link highlighting, smooth theme toggle button
[PASS] **Footer.tsx** — Clean layout, social links, dynamic copyright year
[PASS] **AdminLayout.tsx** — Sidebar navigation, header with logout, responsive design

UI Components

[PASS] **ProjectCard.tsx** — Proper link wrapping, image placeholder handling, tech stack badges
[PASS] **BlogCard.tsx** — Clean card with image, date, category badge
[FAIL] **CommentSection.tsx** — XSS via dangerouslySetInnerHTML on user-submitted comment content
[PASS] **Rating.tsx** — Star rating component with numeric display
[PASS] **SkillBar.tsx** — Animated progress bar with percentage display

Config & Infrastructure

[PASS] **App.tsx** — Clean route definitions, AnimatePresence, proper context wrapping
[PASS] **types/index.ts** — Full TypeScript interfaces for all entities with proper typing
[PASS] **data/mockData.ts** — Complete seed data with all required fields
[MARGINAL] **i18n.ts** — Full en translations but hardcoded lorem ipsum content; no actual other locales configured
[MARGINAL] **index.tsx** — Uses deprecated ReactDOM.render() instead of createRoot()
[PASS] **index.css** — Clean CSS variables, custom utilities, scrollbar styling
[PASS] **tailwind.config.js** — Proper extension of CSS variables, fonts, dark mode config
[MARGINAL] **index.html** — No meta description, lang hardcoded to en, no favicon variety

12. Integration Audit

- [PASS] i18n integration** — Translation keys used consistently across all pages via useTranslation() hook
- [PASS] Theme integration** — All pages read from ThemeContext; CSS variables applied uniformly
- [PASS] DataContext integration** — All pages consume DataContext properly for CRUD and state
- [PASS] Router consistency** — No broken internal links; all <Link> paths match defined routes
- [FAIL] Props to components** — BlogCategoryFilter referenced as prop but never defined/imported
- [PASS] Type consistency** — All entities conform to types/index.ts interfaces
- [MARGINAL] Admin to Public sync** — Admin CRUD updates localStorage; public pages read DataContext. Sync works but stale-closure risk exists when state updates outside React lifecycle

13. Production Readiness & Recommendations

VERDICT: NOT READY FOR PRODUCTION

Critical blockers preventing production deployment:

- Hardcoded admin credentials - anyone who reads source code can log in
- Cross-Site Scripting vulnerability in comment rendering
- No real authentication - a localStorage boolean is not a valid session
- No error boundaries - any unhandled error results in a white screen
- Undefined BlogCategoryFilter component causing broken blog pages

Recommended fixes (in priority order):

#1: Implement proper authentication

Replace hardcoded credentials with JWT-based auth using HttpOnly cookies or OAuth2.0. Implement secure session management with expiry.

#2: Remove XSS vector in CommentSection

Replace dangerouslySetInnerHTML with a sanitized Markdown renderer (e.g., react-markdown + rehype-sanitize).

#3: Add Error Boundaries

Wrap root component and each page with React Error Boundaries to prevent total UI crashes. Log errors gracefully.

#4: Fix BlogCategoryFilter

Either implement the BlogCategoryFilter component or remove its references from Blog.tsx and BlogPost.tsx.

#5: Update to React 18 API

Replace ReactDOM.render() with createRoot() in index.tsx.

#6: Add rate limiting to login

Implement exponential backoff delay or CAPTCHA on failed login attempts.

#7: Implement toast/feedback system

Add success/error notifications for contact form submission and admin CRUD operations.

#8: Add lazy loading for admin routes

Use React.lazy() + Suspense to code-split admin pages from the public bundle.

#9: Add 404 catch-all route

Create a NotFound page and add a catch-all route to handle unknown paths gracefully.

#10: Remove unused React imports

Clean up the 9 files with unused React imports to satisfy strict TypeScript.